DENTIGO

EF Core + SQL Server Final Database Implementation Guide

Clean Architecture • Shared baseline for a two-developer team • October 2026

Use this document as the single agreed reference for entity names, relationships, constraints, and the first migration. It is a reconciled implementation baseline based on the three Dentigo source documents—not a replacement for approved business requirements.

Quick decisions

- Database provider: SQL Server; ORM: Entity Framework Core.
- 11 core entities: User, Patient, Tooth, ClinicalFinding, TreatmentPlan, Procedure, DentalCase, CaseFile, DesignVersion, CaseHistoryEvent, Notification.
- One naming convention and one owner for ApplicationDbContext/migrations.
- Do not create the first migration until all entities/configurations are merged and reviewed.

1. Purpose and source-of-truth

- Use this guide as the agreed implementation baseline for the first database build. It reconciles the current SRS, ERD/class-diagram specification, and Classes Specification document, and includes the SRS clarification that missing radiology files alone must not block Lab submission. If the approved SRS changes, update this guide before changing entities or migrations.
- Priority when documents disagree: (1) SRS_AFTER_EDIT.pdf for required behavior and business rules; (2) Digrames(1).pdf for logical data model and database relationships; (3) Last EDIT Digrame.pdf for class-level detail, retaining its useful attributes when they do not conflict with the first two.

2. Agreed technical and naming conventions

- Database: Microsoft SQL Server. ORM: Entity Framework Core. Architecture: Clean Architecture.
- Entity and class names: singular PascalCase (DentalCase, CaseFile). Property names: PascalCase (DentalCaseId, CreatedAt). Foreign keys: <PrincipalEntityName>Id where practical (PatientId, CreatedByUserId). Collection navigation properties: plural (DentalCases, Files, Notifications).
- Use int identity primary keys for the core entities. Use DateTime in the domain model and configure date/time columns consistently. Required reference data should be non-nullable; optional relationships use nullable FK properties (int?). Use decimal only for numeric monetary values if such features are added later; costs/payments are out of MVP scope.
- Use Fluent API configurations in Infrastructure/Persistence/Configurations, one file per entity. Keep EF Core attributes out of Domain where practical. Do not configure the same relationship in multiple places.
- Store password hashes only; never store plaintext passwords. Do not hard-delete users, cases, files, or case-history records where that would break traceability.

3. Core entities and baseline properties

- User: UserId int PK; FullName nvarchar(150) required; Email nvarchar(254) required unique; PasswordHash required; Role UserRole required; IsActive bit required; CreatedAt datetime required.
- Patient: PatientId int PK; FullName nvarchar(150) required; PhoneNumber nvarchar(30) optional; CreatedAt datetime required; UpdatedAt datetime optional.
- Tooth: ToothId int PK; PatientId int required FK; ToothNumber nvarchar(10) required; Notes nvarchar(max) optional. Tooth numbering format must be consistent across the application.
- ClinicalFinding: ClinicalFindingId int PK; PatientId int required FK; ToothId int required FK; DentalCaseId int? optional FK; Description nvarchar(max) required; CreatedByUserId int required FK to User; CreatedAt datetime required; UpdatedAt datetime? optional.
- TreatmentPlan: TreatmentPlanId int PK; PatientId int required FK; Name nvarchar(150) required; Description nvarchar(max) optional; CreatedByUserId int required FK to User; CreatedAt datetime required.

- Procedure: ProcedureId int PK; TreatmentPlanId int required FK; Name nvarchar(150) required; Description nvarchar(max) optional; SequenceNumber int required; DentalCaseId int? optional FK. Do not add ProcedureId to DentalCase: the optional association is owned by Procedure.DentalCaseId in this baseline.
- DentalCase: DentalCaseId int PK; PatientId int required FK; ToothId int required FK; CreatedByUserId int required FK to User; TreatmentName nvarchar(150) required; Description nvarchar(max) optional; DoctorInstructions nvarchar(max) optional; Status DentalCaseStatus required; RequiresRadiology bit required; ExpectedDueDate datetime? optional; CreatedAt datetime required; UpdatedAt datetime? optional; CancellationReason nvarchar(max) optional.
- CaseFile: CaseFileId int PK; PatientId int required FK; DentalCaseId int required FK; ToothId int? optional FK; UploadedByUserId int required FK to User; FileName nvarchar(255) required; FileType nvarchar(20) required; FileSize bigint required; StoragePath required; FileCategory FileCategory required; UploadedAt datetime required. Keep file metadata in SQL Server; store actual file bytes in configured file storage, not directly in the database unless the team explicitly chooses otherwise.
- DesignVersion: DesignVersionId int PK; DentalCaseId int required FK; CaseFileId int required FK; SubmittedByUserId int required FK to User; VersionNumber int required; ReviewStatus DesignReviewStatus required; SubmittedAt datetime required; ReviewReason nvarchar(max) optional.
- CaseHistoryEvent: CaseHistoryEventId int PK; DentalCaseId int required FK; ActorUserId int? optional FK to User; EventType nvarchar(100) required; Description nvarchar(max) required; PreviousStatus nvarchar(50) optional; NewStatus nvarchar(50) optional; RelatedFileId int? optional FK to CaseFile; RelatedDesignVersionId int? optional FK to DesignVersion; Reason nvarchar(max) optional; CreatedAt datetime required.
- Notification: NotificationId int PK; RecipientUserId int required FK to User; DentalCaseId int? optional FK; NotificationType nvarchar(100) required; Message nvarchar(max) required; CreatedAt datetime required; IsRead bit required.

4. Relationships to configure

- User 1 — many DentalCases through DentalCase.CreatedByUserId.

- User 1 — many TreatmentPlans through TreatmentPlan.CreatedByUserId; User 1 — many ClinicalFindings through ClinicalFinding.CreatedByUserId.
- User 1 — many CaseFiles through CaseFile.UploadedByUserId; User 1 — many DesignVersions through DesignVersion.SubmittedByUserId; User 1 — many CaseHistoryEvents through nullable ActorUserId; User 1 — many Notifications through RecipientUserId.
- Patient 1 — many Teeth; Patient 1 — many DentalCases; Patient 1 — many ClinicalFindings; Patient 1 — many TreatmentPlans.
- Tooth 1 — many DentalCases; Tooth 1 — many ClinicalFindings. Each DentalCase must have exactly one Patient and one Tooth.
- TreatmentPlan 1 — many Procedures. A Procedure may be linked to zero or one DentalCase; a DentalCase can exist without a TreatmentPlan.
- DentalCase 1 — many CaseFiles; DentalCase 1 — many DesignVersions; DentalCase 1 — many CaseHistoryEvents; DentalCase 1 — many Notifications; DentalCase 1 — zero or many ClinicalFindings.
- CaseFile 1 — zero or one DesignVersion in this baseline; enforce a unique index on DesignVersion.CaseFileId if each file can be the design file for at most one version. Each DesignVersion belongs to one DentalCase and references one CaseFile.

5. Required indexes and integrity rules

- Unique index on User.Email.
- Unique composite index on DesignVersion(DentalCaseId, VersionNumber), so a case cannot have duplicate version numbers.
- Recommended unique composite index on Tooth(PatientId, ToothNumber), if the product treats one tooth number as unique per patient.
- Important cross-reference rule: DentalCase.PatientId and DentalCase.ToothId must refer to a tooth belonging to that same patient. Two independent foreign keys do not enforce this. Enforce it with backend validation and, if the team chooses, a composite alternate key/foreign key in SQL Server.
- Validate that ClinicalFinding.PatientId matches the PatientId of ClinicalFinding.ToothId, and that any linked DentalCase belongs to the same patient/tooth as required. Validate file PatientId/ToothId against its DentalCase before saving.

- File constraints: accept only supported formats PDF, JPG, PNG, STL, OBJ; maximum size 25 MB per file. A failed or incomplete upload must not be registered as a valid file.
- Cancellation requires a reason and only a Doctor may cancel a case. Preserve cancelled cases, files, and history. CaseHistoryEvent records cannot be edited/deleted through the application. Deactivated users cannot log in; protect the last active Administrator.
- Do not cascade-delete records that must be retained for traceability. Configure restrictive/no-action delete behavior for core historical relationships; choose delete behavior explicitly for every relationship.

6. Enums and workflow

- UserRole: Doctor, LabTechnician, RadiologyUser, Administrator.
- DentalCaseStatus: Draft, Submitted, InProgress, DesignSubmitted, RevisionRequested, Approved, Completed, Cancelled.
- FileCategory: Clinical, RadiologyImage, RadiologyReport, DentalDesign, OtherPermittedFile.
- DesignReviewStatus: PendingReview, Approved, RevisionRequested, Rejected.
- Use enum properties in C#. For readable SQL Server values, configure HasConversion<string>() and set suitable maximum lengths in each entity configuration. Keep enum names stable after migrations.

SRS clarification / acceptance rule: Treat missing required X-ray images or reports as an incomplete radiology item, not as a submission blocker by itself. Do not implement a validation that rejects Lab submission solely because those files are absent. Other independent validation rules remain applicable.

Radiology is conditional: RequiresRadiology indicates whether radiology is expected for the case; it is not the main case status. Missing radiology images or a radiology report alone must NOT block sending a case to the Lab. The system may show a warning or mark radiology as incomplete, but it must allow submission unless another independent business validation fails. Completing radiology does not complete the DentalCase.

- Approving a design does not automatically complete the DentalCase. The Doctor's review must apply to the latest submitted DesignVersion. Revisions/rejections

must retain the reason and create a new version rather than overwrite an earlier version.

7. Clean Architecture layout

- Dentigo.Domain: Entities and enums only; no DbContext or SQL Server dependency.
- Dentigo.Application: use cases, interfaces/contracts, validation and business operations. Do not put EF Core migrations here.
- Dentigo.Infrastructure: ApplicationDbContext, entity configurations, EF Core provider setup, migrations, and file-storage implementation.
- Dentigo.API: controllers/endpoints, authentication/authorization wiring, dependency injection composition, and HTTP concerns.
- Suggested paths: Dentigo.Domain/Entities; Dentigo.Domain/Enums; Dentigo.Infrastructure/Persistence/ApplicationDbContext.cs; Dentigo.Infrastructure/Persistence/Configurations/<Entity>Configuration.cs; Dentigo.Infrastructure/Persistence/Migrations.

8. Two-person work split without conflicts

- Developer A: User, Patient, Tooth, ClinicalFinding entities and their individual configuration files.
- Developer B: TreatmentPlan, Procedure, DentalCase, CaseFile, DesignVersion, CaseHistoryEvent, Notification entities and their individual configuration files.
- Both developers must use the exact names and property definitions in this guide. Agree on the navigation properties before implementation; do not independently edit ApplicationDbContext or create migrations.
- One designated integrator owns ApplicationDbContext, dependency registration, relationship integration, migration creation, and database update. The other developer reviews the diff and tests the resulting model.
- Use one branch per work item and merge small changes. Do not have both developers run Add-Migration for competing model changes. Create the first migration only after all entities/configurations are merged and the team has reviewed the model.

9. Implementation order

- 1) Create the solution/projects and project references following Clean Architecture dependency direction.
- 2) Create enums and entity classes from Section 3.
- 3) Create one IEntityTypeConfiguration<TEntity> per entity in Infrastructure.
- 4) Add DbSet<TEntity> properties and register configurations in ApplicationDbContext.
- 5) Configure SQL Server connection string via configuration/user secrets; do not commit credentials.
- 6) Review all required/optional FKs, delete behaviors, unique indexes, enum conversions, and the Patient-Tooth integrity rule.
- 7) Build the solution and inspect the EF Core model.
- 8) Create the first migration from the integrated branch, review the generated migration, then update the development database.
- 9) Test unique email, duplicate tooth number (if enabled), unique design version per case, invalid patient/tooth pairing, and specifically verify that missing radiology files alone do NOT block sending a case to the Lab; also test cancellation reason, history preservation, and file-size/type validation.

10. Decisions to revisit only if requirements change

- CaseFile stores PatientId and DentalCaseId and may store ToothId as optional because the ERD requires files to be linkable to patient/case/tooth. These duplicated references require backend consistency validation.
- The ERD and Classes document differ in some property names. This baseline standardizes ClinicalFindingId (instead of FindingId), CaseFileId (instead of FileId), StoragePath (instead of FilePath), ReviewStatus (instead of Status on DesignVersion), RecipientUserId (instead of UserId on Notification), and ActorUserId (instead of PerformedByUserId on CaseHistoryEvent).
- The class document includes DentalCase.ProcedureId, while the ERD/SRS model the optional link as Procedure.DentalCaseId. This guide chooses Procedure.DentalCaseId and omits DentalCase.ProcedureId to avoid storing the same association twice.

- Do not silently change this baseline in code. If a business requirement makes a different model necessary, agree on the change together, update this guide, then create the migration.

Appendix: Source documents

- SRS_AFTER_EDIT.pdf — source for functional/non-functional requirements, roles, workflow, file restrictions, and business rules.
- Digrames(1).pdf — source for logical entities, suggested SQL data types, ERD relationships, integrity rules, and implementation sequence.
- Last EDIT Digrame.pdf — source for class descriptions, attributes, behaviors, and relationship details.